Capítulo 4 · Índices en MongoDB

Manual de teoría — edición escrita desde cero

Destinatarios: personas que conocen la idea de guardar datos, pero que todavía no han diseñado índices en MongoDB.
Entorno de los ejemplos: MongoDB Community 8.0, base `curso_indices`, colección `ventas`.
Finalidad: entender qué problema soluciona cada índice, cómo decidir si crearlo y cómo comprobar sus efectos sin confundir una medición aislada con una garantía.

> En estas páginas cada concepto se introduce mediante una situación concreta. El cuaderno de prácticas utiliza los mismos nombres de base de datos y campos. Ningún ejemplo requiere comprender de antemano árboles de búsqueda o los detalles internos del motor.

---

1. Antes de hablar de índices: ¿dónde están nuestros datos?

Imaginemos una librería que recibe pedidos por internet y en tienda. Cada venta tiene un número, un cliente, una fecha y un importe. Cuando el personal busca la venta 198.765 no quiere leer cientos de miles de operaciones una por una: necesita localizar esa venta y mostrarla.

En una base de datos relacional solemos imaginar una tabla. En MongoDB hablamos de **documentos**: unidades de información que se parecen a un objeto JSON. Un conjunto de documentos se guarda en una **colección**, y las colecciones viven dentro de una **base de datos**.

Un documento didáctico podría ser:

```
{
  numero: 198765,
  clienteId: 901,
  estado: "pagada",
  fecha: ISODate("2025-05-16T00:00:00Z"),
  importe: 72.50,
  etiquetas: ["web", "promocion"],
  descripcion: "compra habitual"
}
```

La diferencia respecto a un texto JSON corriente es que `ISODate(...)` representa una fecha real en MongoDB: eso permite filtrar por rangos de fechas correctamente. En una colección, cada documento tiene además un identificador `_id` único. MongoDB crea automáticamente un índice sobre ese identificador.

Qué debe entenderse antes de continuar: documento no significa archivo en el disco; es una unidad de datos. Colección no significa carpeta de Windows; es un conjunto lógico de documentos. Y un índice no es otra colección con una copia completa de las ventas.

Una primera consulta

En `mongosh`, el intérprete de comandos de MongoDB, escribimos:

```
use curso_indices
db.ventas.findOne({numero: 198765})
```

`findOne` solicita un documento, mientras que el objeto entre llaves expresa el criterio: el campo `numero` debe ser igual a `198765`. El resultado será el documento encontrado o `null` si no existe.

Es importante distinguir **lo que pedimos** de **cómo MongoDB lo encuentra**. Esa consulta describe *qué* buscamos, pero no obliga por sí misma a utilizar un índice. MongoDB decide el plan de ejecución según los índices disponibles, los filtros y otra información.

Una analogía que tiene límites

Un libro impreso contiene páginas y un índice alfabético. Si buscamos «autenticación», consultamos el índice, localizamos una página y la abrimos. En una colección también podemos construir una estructura auxiliar que relaciona valores con documentos. Pero la analogía termina ahí: un índice de MongoDB consume espacio, se modifica cuando cambian los datos y puede participar en búsquedas por rango y ordenaciones. No es solamente una lista de palabras.

Comprueba que entiendes la diferencia: si la librería tiene 200.000 ventas y alguien conoce el número exacto, ¿tiene sentido examinar en detalle las 200.000? Tal vez con pocas ventas no importe; con cientos de miles el coste puede ser evidente. Un índice es una herramienta para cambiar ese trabajo.

---

2. Qué sucede cuando NO existe un índice útil

Supongamos que todos los documentos están disponibles, pero el servidor carece de un índice sobre `numero`. Para encontrar una venta concreta, una estrategia posible consiste en revisar documentos de la colección y comprobar si cada número coincide. MongoDB llama `COLLSCAN` a una etapa que recorre documentos de una colección.

No significa siempre «un error». Si una colección tiene diez documentos, recorrerlos puede ser más sencillo que mantener un índice adicional. El problema aparece cuando la búsqueda devuelve muy pocos resultados pero el motor debe examinar muchísimos candidatos.

Cardinalidad, selectividad y trabajo

Considera dos preguntas diferentes:

1. «¿Cuál es la venta número 198.765?». Esperamos una sola coincidencia.
2. «¿Qué ventas se encuentran en estado “pagada”?». Podrían existir decenas de miles.

En el primer caso el filtro suele ser **muy selectivo**: deja pasar una proporción diminuta de la colección. En el segundo puede ser **poco selectivo**: muchísimas ventas comparten el mismo valor.

La **cardinalidad** de un campo se refiere al número de valores distintos que aparecen. `numero` tendrá habitualmente muchos valores distintos; `estado` quizá solo cuatro. Cardinalidad y selectividad se relacionan, pero no son equivalentes: un campo con muchos valores distintos no garantiza que *cada* consulta sobre él devuelva pocas filas.

Si hay un millón de ventas y 250.000 pagadas, un índice exclusivo sobre `estado` quizá obligue a recorrer muchísimas claves y documentos. El planificador puede preferir otro camino. **No debemos enseñar que todo `IXSCAN` supera automáticamente a todo `COLLSCAN`.**

Qué significa «examinar»

Hay que diferenciar:

- **Documentos devueltos:** los que recibe el cliente después de aplicar el criterio.
- **Documentos examinados:** los que el plan necesita leer para resolver la consulta.
- **Entradas de índice examinadas:** referencias de la estructura indexada que el plan recorre.

Una consulta puede devolver un documento después de examinar un millón. Y puede devolver el mismo documento examinando una sola entrada de índice y un solo documento.

Coste y memoria caché

Dos consultas aparentemente idénticas pueden tardar tiempos diferentes. La primera podría necesitar leer datos que todavía no están en memoria; la segunda quizá encuentre páginas ya cargadas. También influyen CPU, carga del equipo, límites de recursos y otras actividades.

Por tanto, cuando comparemos alternativas, registraremos primero **qué plan eligió MongoDB y cuánto trabajo contabilizó**. Solo después hablaremos de milisegundos. Atribuir una mejora al índice porque una ejecución duró menos sería una explicación insuficiente.

Pregunta para razonar: si una búsqueda devuelve 1 venta tras examinar 200.000 documentos, ¿qué dato expresa mejor el desperdicio de trabajo: un tiempo de 90 ms o el contador de 200.000 documentos examinados? Explica por qué.

---

3. Qué es físicamente un índice y qué precio tiene

Un índice es una estructura de datos mantenida por el motor para facilitar ciertas operaciones. En los índices ordinarios de MongoDB, el concepto esencial es una organización de claves ordenadas que permite localizar valores o intervalos sin recorrer la colección completa.

Podemos representarlo de manera muy simplificada:

```
Valores del índice numero
  1001  → referencia al documento de la venta 1001
  1002  → referencia al documento de la venta 1002
  1003  → referencia al documento de la venta 1003
  ...
198765  → referencia al documento buscado
```

Atención: ese dibujo es conceptual, no el formato real de almacenamiento. WiredTiger usa estructuras B-tree y mecanismos internos de páginas y memoria; aquí solo necesitamos entender que las claves están organizadas para evitar un recorrido indiscriminado.

Un índice también cuesta

Crear `{numero: 1}` aporta una ventaja potencial a las consultas por número, pero introduce tres costes:

1. **Espacio adicional.** Hay claves, referencias y metadatos que guardar.
2. **Escrituras más caras.** Cuando insertamos una venta, quizá haya que insertar una nueva clave en varios índices.
3. **Mantenimiento.** Si cambia un campo indexado, MongoDB debe mantener la consistencia del índice con los documentos.

Imagínate una librería con diez índices diferentes. Cada venta nueva puede exigir mantenimiento sobre esos diez índices. Por eso no es sensato crear uno por cada pregunta hipotética. Primero identificamos las consultas frecuentes y después diseñamos los índices adecuados.

Orden ascendente y descendente

`{numero: 1}` significa que `numero` es la clave del índice, en sentido ascendente. `{fecha: -1}` indica orden descendente para la fecha. En un índice compuesto el orden de las claves es especialmente importante, porque condiciona qué filtros y ordenaciones pueden aprovechar la estructura.

En consultas de igualdad sobre una única clave, un índice ascendente y uno descendente suelen permitir el mismo tipo de búsqueda. No debemos atribuir importancia mágica al signo `1`.

El índice implícito _id

Toda colección estándar tiene un índice único sobre `_id`. Es útil para obtener un documento a partir de su identificador interno, pero **no acelera automáticamente** una búsqueda por `numero`,

`clienteId` o `fecha`. Que una colección «tenga un índice» no significa que «esté indexada para nuestras consultas».

Una decisión técnica razonable

Antes de crear cualquier índice formula tres preguntas: ¿qué operación necesita acelerarse?, ¿qué proporción de documentos devuelve?, ¿cuál es el coste adicional de escrituras y almacenamiento? Si no podemos responder, todavía no tenemos fundamento suficiente para crear el índice.

---

4. Primer índice útil: buscar una venta por su número

En la colección docente cada venta contiene `numero`. Si la aplicación busca muchas veces por ese número, creamos:

```
db.ventas.createIndex({numero: 1})
```

La llamada devuelve el nombre del índice creado o del índice ya existente. No devuelve directamente el número de documentos que la consulta examinará. Ese dato se observa utilizando `explain()`.

Antes y después sin cambiar la pregunta

Para evaluar el efecto necesitamos preservar **la misma consulta**:

```
db.ventas.find({numero: 198765}).explain("executionStats")
```

Si ejecutamos esto antes del índice podremos encontrar una etapa `COLLSCAN`. Después de crearlo podremos encontrar `IXSCAN`. En algunos motores o planes el índice puede aparecer anidado dentro de `FETCH`, por lo que hay que mirar el árbol de etapas, no solo la primera palabra.

Supongamos que el resultado resumido fuera:

Medida	Antes	Después
Coincidencias	1	1
Documentos examinados	200.000	1
Claves examinadas	0	1
Acceso dominante	COLLSCAN	IXSCAN

Esto es un **ejemplo pedagógico de lectura**, no una promesa de cifras para cualquier ordenador. La práctica mostrará los números reales del laboratorio. Lo importante es que hemos respondido la misma pregunta obteniendo la misma venta con un volumen de trabajo distinto.

Índice único

Si el negocio exige que ninguna venta comparta el mismo número, podemos declarar esa regla con:

```
db.ventas.createIndex({numero: 1}, {unique: true})
```

Un índice `unique` no significa «más rápido que cualquier otro índice». Significa que MongoDB impide duplicados de la clave indexada. Es una restricción de integridad además de una

estructura de acceso. Si ya hay dos documentos con el mismo número, crear el índice único fallará hasta resolver esos duplicados.

Qué no demuestra el ejemplo

No hemos demostrado que el índice reduzca la duración de todas las consultas ni que sea el mejor para ordenar ventas por fecha. Únicamente hemos demostrado que **esta búsqueda concreta por `numero`** puede aprovechar una estructura apropiada.

---

5. Cómo leer explain() sin perderse en el JSON

La salida completa de `explain()` puede ser muy extensa. El objetivo de un principiante no es memorizar todas las propiedades. Debe aprender a formular preguntas que la salida pueda contestar.

Usaremos el siguiente orden de lectura:

1. **¿Cuántos resultados devolvió?** Busca `executionStats.nReturned`.
2. **¿Cuánto trabajo hizo?** Busca `totalDocsExamined` y `totalKeysExamined`.
3. **¿Qué estrategia siguió?** Localiza `COLLSCAN`, `IXSCAN`, `FETCH` y, si existe, `SORT` en el plan.
4. **¿Cuánto tardó aquí?** Revisa `executionTimeMillis`, recordando que una medición no es un benchmark.

Por qué aparece FETCH

Un índice puede contener `numero` y una referencia al documento, pero la consulta solicita campos como `descripcion` o `importe`. MongoDB puede buscar el número por el índice (`IXSCAN`) y después leer el documento (`FETCH`). No es una contradicción: usar un índice y leer el documento suelen ser operaciones complementarias.

Por qué puede aparecer SORT

Si pedimos las diez ventas más recientes y el índice usado no proporciona el orden requerido, el plan puede necesitar una etapa de ordenación adicional. Esa etapa no implica necesariamente un problema grave, pero podría consumir trabajo y memoria. Más adelante veremos cómo un índice compuesto puede ayudar.

El error de mirar solo executionTimeMillis

Imagina estos dos resultados:

- Consulta A: 300.000 documentos examinados, 32 ms.
- Consulta B: 6 documentos examinados, 46 ms.

No tenemos permiso para concluir que A es una mejor estrategia por ser más corta en esa única medición. Tal vez B cargó páginas por primera vez, tal vez hubo variaciones de CPU, o quizá la colección es muy pequeña. Necesitamos interpretar tanto trabajo como tiempo y repetir mediciones si queremos hacer una comparación de rendimiento seria.

Técnica de depuración

Cuando no aparece el índice esperado, comprueba antes de nada: nombre exacto del campo, tipo de dato, filtros usados, índices existentes y si la consulta es la que pretendíamos ejecutar. Un error tan simple como buscar `"198765"` (texto) cuando `numero` guarda `198765` (número) puede devolver cero resultados y arruinar la comparación.

---

6. Índices compuestos: por qué importa el orden de los campos

Una consulta frecuente de la librería es: «Quiero las ventas del cliente 901, comenzando por las más recientes». Hay dos operaciones juntas: filtrar por cliente y ordenar por fecha.

Podemos crear un índice compuesto:

```
db.ventas.createIndex({clienteId: 1, fecha: -1})
```

Compuesto significa que cada entrada usa más de un campo. No son dos índices separados: es una estructura con una clave formada, primero, por `clienteId` y después por `fecha`.

Piensa en un archivador ordenado por apellido y, dentro de cada apellido, por fecha. Buscar todo lo de «García» es sencillo. Buscar todos los registros de «diciembre», sin saber el apellido, no puede aprovechar de la misma manera ese orden.

Prefijo izquierdo

En `{clienteId: 1, fecha: -1}` el primer campo es `clienteId`. El índice puede ser útil para filtrar por `clienteId`; también para filtros que utilizan `clienteId` y `fecha` juntos. Una consulta que filtre **solo por `fecha`** no obtiene automáticamente el mismo beneficio de búsqueda selectiva.

El «prefijo izquierdo» no es una restricción de la sintaxis de consultas. Puedes consultar cualquier campo sin índice. Es una idea sobre la disposición de las claves y el alcance de un índice compuesto.

Índices compuestos y ordenaciones

Una consulta como:

```
db.ventas.find({clienteId: 901}).sort({fecha: -1}).limit(5)
```

pide solo cinco documentos, pero también impone un orden. Si el índice compuesto es adecuado para ese filtro y esa ordenación, MongoDB puede recorrer las claves de manera útil y evitar una etapa `SORT` separada.

No demos por supuesto que siempre sucederá: se debe mirar el plan real.

Equality, Sort, Range: la estrategia ESR

Una heurística muy conocida propone organizar los campos de un índice compuesto según el patrón **igualdad**, **ordenación** y **rango** (ESR). Por ejemplo, si filtramos por tienda, ordenamos por fecha y además imponemos una condición de rango sobre importe, podría ser razonable estudiar:

```
db.ventas.createIndex({tienda: 1, fecha: -1, importe: 1})
```

La estrategia ESR no es una ley infalible. En algunas consultas selectivas, un rango antes de la ordenación puede reducir mucho el trabajo; otras restricciones de consulta pueden alterar la decisión. El objetivo es aprender a proponer candidatos, contrastar los planes y elegir con datos.

Elegir entre dos propuestas

Para «ventas pagadas de un cliente, de la más reciente a la más antigua», compara conceptualmente:

- `{estado: 1, clienteId: 1, fecha: -1}`.
- `{clienteId: 1, fecha: -1}`.

El primero incluye el estado; el segundo tiene una clave más corta y quizá ya satisface muchas operaciones por cliente. La elección depende de las consultas reales y la distribución de los datos. Añadir campos «por si acaso» aumenta coste.

---

7. Índices sobre listas: multikey

Los documentos de venta contienen un campo `etiquetas` que puede guardar varios valores:

```
{numero: 198765, etiquetas: ["web", "promocion"]}
```

Si buscamos ventas con la etiqueta `"promocion"`, MongoDB puede utilizar un índice sobre ese campo:

```
db.ventas.createIndex({etiquetas: 1})
db.ventas.find({etiquetas: "promocion"})
```

Cuando se indexa un campo que contiene un array, MongoDB crea un índice de tipo **multikey**. Es decir, el índice permite encontrar un documento por valores presentes en la lista.

Lo que sorprende al principiante

La condición `{etiquetas: "promocion"}` no exige que el array sea exactamente igual al texto. En este contexto, coincide con documentos cuyo array contiene ese elemento. En cambio, consultar una lista completa o imponer varias condiciones al mismo array requiere comprender con cuidado la semántica de coincidencias y operadores como `$elemMatch`.

No confundas «hay varias entradas de índice» con «el documento se duplica». La colección sigue almacenando una venta; lo que se multiplica son las claves que ayudan a encontrarla.

Limitaciones prácticas

Los índices multikey tienen restricciones de diseño: no todos los índices compuestos admiten libremente múltiples campos de tipo array en un mismo documento. También pueden complicar ordenaciones y consultas cubiertas. Por eso conviene partir de un ejemplo simple que funciona y estudiar las restricciones antes de usar arrays anidados complejos.

Cuándo tiene sentido

Son adecuados para etiquetas, categorías asignadas a un producto, identificadores de participantes y otros campos donde un documento posee varios valores. No reemplazan el análisis de consultas: si la aplicación nunca busca por las etiquetas, no necesitamos mantener el índice.

---

8. Búsqueda de texto: palabras, no igualdad literal

En una consulta de igualdad:

```
db.ventas.find({descripcion: "compra habitual"})
```

solo buscamos documentos cuyo campo coincida con ese valor. Pero quizá el personal quiere encontrar descripciones que mencionan «reclamación», aunque el texto completo sea mucho más largo. Para ese caso se estudia un **índice de texto**:

```
db.ventas.createIndex({descripcion: "text"})
db.ventas.find({$text: {$search: "reclamacion"}})
```

En un índice `text` clásico, MongoDB analiza términos y ofrece búsqueda por palabras según reglas del analizador, idioma y normalización. Eso **no equivale** a comparar cadenas completas, ni a una búsqueda arbitraria de subcadenas ni a disponer de un buscador semántico inteligente.

Tres preguntas importantes

¿Por qué «compra» no es igual a «compra habitual»? Porque una condición de igualdad trata el texto como valor completo; `$text` consulta términos indexados.

¿Por qué algunas palabras frecuentes no aparecen como imaginábamos? El análisis de texto puede aplicar reglas de idioma, stemming y palabras vacías. Es imprescindible elegir el idioma correcto y probar con descripciones reales.

¿Qué significa relevancia? Se puede proyectar una puntuación de texto mediante `{$meta: "textScore"}` y ordenar con ella, pero esa puntuación es propia del mecanismo de búsqueda; no equivale a una calificación universal de calidad del resultado.

El motor de búsquedas avanzado de MongoDB Atlas ofrece otras capacidades; no debemos confundirlo con el índice `text` clásico disponible en Community.

---

9. Índices geoespaciales: una coordenada necesita contexto

La librería dispone de puntos de venta. Para localizar los próximos a un lugar necesitamos almacenar coordenadas con un formato reconocido y crear un índice geoespacial.

Un punto GeoJSON se representa así:

```
{type: "Point", coordinates: [-3.7038, 40.4168]}
```

En GeoJSON, para este uso, la primera cifra es **longitud** y la segunda **latitud**. Intercambiarlas puede trasladar el punto a otra zona del mundo.

Creamos un índice:

```
db.ventas.createIndex({ubicacion: "2dsphere"})
```

Con `2dsphere` podemos trabajar con geometrías de superficie esférica y realizar consultas como `$near`, respetando sus requisitos.

¿Qué ocurre si no hay índice?

No todas las operaciones geoespaciales funcionan igual sin un índice. Para consultas de proximidad con `$near`, el índice geoespacial apropiado resulta esencial. Si escribimos geometrías inválidas, coordenadas invertidas o tipos incorrectos, puede fallar la indexación o la consulta.

Unidad de distancia y precisión

Las distancias expresadas por ciertos operadores geoespaciales se interpretan en metros al trabajar con GeoJSON y `2dsphere`. No confundas metros con grados ni supongas que una coordenada equivale a una calle exacta: la precisión depende de los datos de origen.

Ejercicio de pensamiento

Una consulta que pide «tiendas a menos de 3 km» es distinta de «ventas de una tienda concreta». La primera pregunta es espacial; la segunda es una igualdad sobre `tienda`. Cada una sugiere estrategias de índice diferentes.

---

10. Hashed, unique y TTL: tres propósitos que no conviene mezclar

Índice hashed

Un índice sobre un valor transformado por hash:

```
db.ventas.createIndex({idGrupo: "hashed"})
```

es adecuado para ciertas búsquedas por igualdad y para casos de distribución de datos, por ejemplo al diseñar claves de fragmentación. **No conserva el orden numérico natural del campo**, por lo que no debemos presentarlo como el índice ideal para una condición por rangos `$gte` / `$lte`.

Índice unique

`unique` añade una regla de integridad: impide valores duplicados en la clave que controla. No sustituye la validación de otros campos ni se debe usar para un valor de negocio cuya repetición sea legítima.

Si dos clientes pueden realizar compras del mismo importe, sería incorrecto exigir `unique` sobre `importe`. En cambio, el número oficial de venta podría requerir unicidad.

La práctica insertará dos documentos con el mismo correo electrónico en una colección aislada: el segundo recibirá el error de clave duplicada `E11000`. Ese error es una demostración válida de la restricción.

Índice TTL: caducidad automática

TTL significa **time to live**: tiempo de vida. Un índice TTL permite que MongoDB elimine documentos después de cierta fecha o de un intervalo calculado desde un campo de fecha.

Para una sesión que posee fecha exacta de expiración:

```
db.sesiones.createIndex({expiraEn: 1}, {expireAfterSeconds: 0})
```

Si un documento contiene `expiraEn` en el pasado, puede ser eliminado por el monitor TTL. **No ocurre necesariamente en el instante exacto**: un proceso en segundo plano revisa expiraciones periódicamente. Nunca diseñes una regla de autorización que dependa exclusivamente de que el monitor ya haya borrado un documento.

Diferencia en una frase

- Hashed: cómo acceder o distribuir por una clave transformada.
- Unique: qué duplicados están prohibidos.
- TTL: qué documentos deben caducar y eliminarse con el tiempo.

Son ideas distintas y no deben enseñarse como variantes intercambiables de «un índice más rápido».

---

11. Consultas cubiertas y el coste de recuperar documentos

Supongamos que un informe solo necesita `clienteId` e `importe`. Si existe un índice compuesto que contiene los campos requeridos por el filtro y por la proyección, puede darse una **consulta cubierta**: MongoDB resuelve la consulta leyendo claves de índice, sin recuperar documentos de la colección.

Ejemplo conceptual:

```
db.ventas.createIndex({clienteId: 1, importe: 1})
db.ventas.find(
  {clienteId: 901},
  {_id: 0, clienteId: 1, importe: 1}
)
```

Se excluye explícitamente `_id` con `_id: 0` porque, de lo contrario, el resultado normalmente solicita ese campo y puede impedir la cobertura si no está en el índice.

Cómo verificarla

Observa `totalDocsExamined` en `explain("executionStats")`. Si es **0**, el plan no necesitó examinar documentos para esa consulta. Aun así puede recorrer múltiples claves. Una consulta cubierta no significa «coste cero».

Qué no debe hacerse

No conviene ampliar todos los índices con decenas de campos solo para conseguir cobertura: cada campo extra ocupa espacio y aumenta las tareas de mantenimiento. La cobertura se justifica en patrones de consulta repetidos y relevantes.

---

12. Diseñar, comprobar y mantener: un método para toda la vida

Ya conocemos los tipos fundamentales. El siguiente paso no es aprender otra orden, sino **decidir responsablemente**.

Paso A: escribe las consultas reales

No diseñes índices a partir de la lista de campos de un documento. Anota preguntas de negocio: «una venta por número», «las cinco ventas recientes de un cliente», «ventas en rango de fecha por tienda», «buscar reclamaciones», «documentos expirados».

Para cada pregunta, escribe filtro, ordenación, proyección y volumen esperado. A continuación identifica cuáles se ejecutan con mayor frecuencia y cuáles tienen mayor impacto cuando tardan.

Paso B: formula una hipótesis comprobable

Ejemplo: «Si creo un índice sobre `numero`, la búsqueda de una venta por su número examinará menos documentos». La hipótesis es concreta. No afirma que MongoDB sea siempre más rápido en cualquier escenario.

Paso C: mide la misma consulta antes y después

Registra los índices existentes, ejecuta `explain("executionStats")`, anota resultados, crea el índice y repite **sin cambiar el filtro**. Si modificas al mismo tiempo el volumen de ventas, la condición y el límite, ya no puedes atribuir una diferencia al índice.

Paso D: contabiliza el coste

Comprueba el tamaño total de índices con herramientas como `stats()` o `collStats` cuando corresponda y recuerda que las escrituras deben mantenerlos. El índice mejor para una lectura puede ser innecesario en una colección con muchas escrituras y pocos accesos a ese patrón.

Paso E: revisa después de cambiar la aplicación

Las consultas evolucionan. Un índice que antes era indispensable puede quedar sin uso y otro puede necesitar un orden distinto de claves. Existe instrumentación para estudiar estadísticas de acceso, pero los contadores se interpretan dentro de una ventana de observación: un cero no demuestra por sí solo que el índice nunca será útil.

Lo que debes poder explicar sin memorizar

Si otra persona te pregunta «¿por qué hemos creado este índice?», deberías ser capaz de responder:

> Porque la aplicación localiza ventas por ese campo; antes el plan examinaba prácticamente toda la colección, y con el índice puede recorrer las claves apropiadas y recuperar muy

> pocos documentos. A cambio consumimos espacio y trabajo de mantenimiento, que hemos considerado aceptables.

Esa explicación, acompañada de una ejecución real, vale mucho más que recitar una lista de tipos de índices.

---

13. Índices parciales: indexar solo lo que interesa

Una colección puede contener millones de ventas históricas, pero el equipo de atención solo consulta a menudo las que están pendientes de tramitar. ¿Debemos mantener un índice especializado sobre todas las ventas de hace diez años para resolver las consultas sobre pedidos pendientes?

Un **índice parcial** permite incluir únicamente los documentos que satisfacen una condición. Un ejemplo simplificado:

```
db.ventas.createIndex(
  {fecha: -1},
  {
    name: "ix_pendientes_fecha",
    partialFilterExpression: {estado: "pendiente"}
  }
)
```

El índice contiene claves de fecha solo para ventas cuyo estado sea `"pendiente"`. Su atractivo principal no es una promesa absoluta de velocidad, sino reducir el número de entradas que deben mantenerse para un determinado patrón de uso.

Una consulta que podría aprovecharlo

```
db.ventas.find({estado: "pendiente"})
  .sort({fecha: -1})
  .limit(10)
```

La consulta comunica explícitamente la condición del índice parcial. El planificador puede considerarlo al saber que ese subconjunto es suficiente para producir resultados correctos.

Una consulta que NO debe perder datos

```
db.ventas.find({})
  .sort({fecha: -1})
  .limit(10)
```

Esta segunda pregunta significa «las diez ventas más recientes sin importar el estado». El índice parcial anterior **no representa todos los estados**, de modo que usarlo como única fuente podría omitir documentos. MongoDB no debe seleccionar un índice parcial si eso ocasiona un resultado incompleto.

Esta distinción enseña una regla de enorme importancia: un índice solo es útil si **contiene la información que la consulta necesita**.

Diferencia con un índice sparse

Un índice disperso o *sparse* puede excluir documentos que no contienen un campo. Un índice parcial es más expresivo: puede incluir documentos según condiciones, incluso sobre campos distintos de la clave indexada. No son sinónimos, aunque algunos comportamientos se solapan.

Pregunta de razonamiento

Si el estado de una venta cambia de `"pendiente"` a `"pagada"`, ¿qué sucede con la entrada correspondiente en un índice parcial que solo incluye pendientes? El motor debe actualizar el índice para reflejar la nueva condición. Por eso el índice sigue teniendo un coste de mantenimiento.

---

14. Un índice no es una orden: el planificador puede escoger otro

A veces creamos el índice que creemos perfecto, ejecutamos la consulta y descubrimos que MongoDB ha utilizado otro distinto. Es frustrante si esperábamos que `createIndex` obligara al motor a escogerlo.

Pero **crear un índice no equivale a imponer su uso**. MongoDB dispone de un planificador que considera alternativas según la forma de la consulta y las estructuras disponibles.

El laboratorio mental de dos índices

Imagina que existen:

```
{clienteId: 1}
{clienteId: 1, fecha: -1}
```

La consulta:

```
db.ventas.find({clienteId: 901}).sort({fecha: -1}).limit(5)
```

puede obtener mejores condiciones con el índice compuesto, porque atiende filtro y ordenación. En otra consulta por cliente que necesita todos los documentos sin orden, el índice simple también puede resultar interesante.

Qué hace hint()

`hint()` permite señalar un índice específico para una prueba o una decisión explícita:

```
db.ventas.find({clienteId: 901})
  .hint({clienteId: 1})
  .explain("executionStats")
```

El valor de `hint` puede ser un patrón de claves o el nombre exacto de un índice. Si se indica uno inexistente, puede aparecer un error. Y si el índice existe pero no es apropiado, el rendimiento puede empeorar.

Lo usamos para **comparar hipótesis**, no como remedio automático a que el servidor no elija nuestra opción.

Una enseñanza que evita malos hábitos

No hay una regla «más índices = más rendimiento». Añadir índices similares puede elevar el trabajo en escrituras sin ofrecer mejoras relevantes. En algunos casos conviene eliminar redundancias, siempre tras estudiar las consultas que el índice atendía.

---

15. Cuando el tipo del campo cambia, cambia la consulta

MongoDB permite almacenar distintos tipos de valores: números, cadenas de texto, fechas, objetos, listas. Eso aporta flexibilidad, pero obliga a prestar atención a cómo se escriben consultas y cómo se construyen los índices.

Dos valores que se parecen pero no significan lo mismo

```
{numero: 12345}
{numero: "12345"}
```

El primero es un número; el segundo es texto. Aunque la pantalla muestre los mismos dígitos, las comparaciones y el orden de los valores **no deben suponerse equivalentes**.

Por ejemplo, una condición por rango sobre números tiene sentido aritmético:

```
db.ventas.find({importe: {$gte: 100, $lt: 200}})
```

Si `importe` se guardara como texto en algunos documentos, un principiante podría sorprenderse con los resultados o con una distribución distinta de los tipos.

El ejemplo clásico de una fecha

Una fecha guardada como `ISODate("2025-07-01")` admite filtros temporales naturales. Una fecha almacenada como texto requiere razonar sobre el formato y no debe tratarse ingenuamente como fecha real.

Para saber qué has almacenado, inspecciona un documento y sus valores. No comiences a «optimizar» índices cuando el problema básico es que los datos tienen tipos inesperados.

Qué debe hacer el desarrollador

Definir un esquema de datos comprensible, mantener tipos coherentes, validar las entradas de la aplicación y probar las consultas sobre datos representativos. El índice no corrige por sí mismo los errores de modelado.

---

16. Índices y ordenaciones: el coste oculto de TOP N

Una página de administración muestra las diez ventas más recientes. La consulta puede parecer sencilla:

```
db.ventas.find({tienda: "Madrid"})
  .sort({fecha: -1})
  .limit(10)
```

Pero `limit(10)` por sí solo no garantiza que el motor lea únicamente diez documentos. Primero necesita localizar qué documentos cumplen el filtro **y** cuáles son los más recientes.

Dos situaciones distintas

Sin ordenación útil en un índice: el servidor quizá deba identificar candidatos y ordenar, antes de decidir los diez primeros. Según el plan y el volumen, eso puede implicar más trabajo del que sugiere `limit`.

Con un índice compuesto compatible: si las claves están organizadas por tienda y, dentro de cada tienda, por fecha descendente, el motor puede recorrer las primeras entradas necesarias sin realizar una ordenación separada.

```
db.ventas.createIndex({tienda: 1, fecha: -1})
```

Por qué no es lo mismo un filtro y un sort

El filtro decide **qué registros cumplen** una condición. La ordenación decide **en qué secuencia aparecen**. Son requisitos distintos, y el índice debe considerarlos juntos.

Pregunta técnica de transferencia

Una aplicación muestra las diez ventas más recientes de *cualquier* tienda. ¿Basta el índice `{tienda:1,fecha:-1}` para optimizar del mismo modo la ordenación global por fecha? No está garantizado: las claves están agrupadas primero por tienda. El prefijo de igualdad que permitía aprovechar la fecha ha desaparecido.

---

17. Cómo observar el tamaño y el uso de los índices

Un índice puede ser muy útil para una lectura y, al mismo tiempo, consumir una cantidad apreciable de almacenamiento y memoria. La decisión de mantenerlo no termina cuando la consulta funciona.

Inventario inicial

```
db.ventas.getIndexes()
```

Esta orden permite enumerar nombres, claves y ciertas opciones. Si aparecen diez índices sobre la misma colección, eso no constituye por sí solo un problema: necesitamos entender qué consultas atiende cada uno.

Tamaño de las estructuras

Las estadísticas de una colección pueden proporcionar información sobre almacenamiento e índices. Dependiendo de la versión y de los permisos de la cuenta, se pueden consultar campos relacionados con `indexSizes` y `totalIndexSize` en `stats()`.

```
db.ventas.stats()
```

No confundas el espacio de los documentos con el espacio de los índices. Tampoco supongas que el tamaño informado es idéntico en cualquier servidor: hay compresión, diferencias de almacenamiento y otros factores.

Estadísticas de acceso

MongoDB permite consultar estadísticas de acceso a índices mediante `$indexStats`:

```
db.ventas.aggregate([{$indexStats: {}}])
```

Estos contadores se interpretan en el contexto del servidor y su periodo de observación. Un índice con cero accesos durante una sesión de clase no es necesariamente inútil: quizá atienda una tarea de fin de mes que todavía no se ejecutó.

Una conclusión responsable

Para valorar un índice hay que combinar **beneficio medido**, **coste de almacenamiento**, **coste de actualización** y **frecuencia del patrón de consulta**. Ninguna cifra aislada sustituye a ese análisis.

---

18. Ocultar un índice para estudiar el efecto de retirarlo

Supongamos que sospechamos que un índice antiguo ya no se utiliza. Eliminarlo directamente obliga a reconstruirlo si después descubrimos que seguía siendo importante.

MongoDB permite **ocultar un índice** al planificador de consultas. Esto significa que las consultas no pueden seleccionarlo para sus planes mientras permanezca oculto.

Ejemplo en una colección de laboratorio:

```
db.ventas.hideIndex("ix_cliente_fecha")
```

A continuación repetimos las consultas por cliente y observamos los planes. Si la situación empeora, podemos volver a hacerlo visible:

```
db.ventas.unhideIndex("ix_cliente_fecha")
```

¿Desaparece el índice de disco?

No. Ocultarlo **no lo elimina**, y no deja de consumir automáticamente recursos de mantenimiento o almacenamiento. Si el índice es único, la restricción de unicidad se mantiene; si es TTL, sigue pudiendo caducar documentos.

Limitaciones

No se puede ocultar el índice obligatorio `_id`. Tampoco podemos forzar mediante `hint` un índice oculto. La práctica debe comprobar siempre cuál está oculto antes de hacer pruebas y restaurarlo al terminar.

Por qué es más seguro que borrar por ensayo

Es una herramienta de diagnóstico reversible. Enseña a trabajar con hipótesis: «Creo que este índice ya no es necesario». La hipótesis se contrasta observando cambios reales en los planes y en la carga representativa, no eliminando estructuras irreversiblemente.

---

19. El coste de construir índices sobre muchos documentos

Crear un índice no es solo escribir una línea y obtener una mejora inmediata en todas las aplicaciones. Para construirlo, MongoDB debe procesar los documentos existentes y preparar la estructura.

Colección pequeña frente a colección grande

Sobre diez documentos, el trabajo de construir un índice es diminuto. Sobre millones, consume CPU, entrada/salida de datos, memoria y espacio. Los mecanismos internos y la disponibilidad de escrituras dependen de la versión y de la configuración del despliegue; no simplificaremos el proceso diciendo que «todo se bloquea» o que «no afecta en nada».

No ejecutar cambios indiscriminados en producción

Crear y borrar índices en una base de datos utilizada por clientes puede alterar el uso de recursos y los planes de consulta. En una organización real se evalúan ventanas, carga, supervisión y posibilidades de reversión.

Caso de aprendizaje

Imagina que creas veinte índices en un sistema que recibe mil ventas por segundo. Cada venta nueva no solo añade un documento: también obliga a mantener las entradas de los índices afectados. Esa carga puede convertirse en un problema aunque las búsquedas individuales resulten rápidas.

El criterio correcto

Un índice debe justificar su existencia por consultas de negocio o restricciones de integridad, no por estar disponible como opción. El número adecuado de índices depende de los patrones de lectura y escritura, no de una cifra universal.

---

20. Diseñar índices cuando cambia la manera de buscar

Antes de terminar, vamos a resolver un pequeño caso completo sin ejecutar ningún comando:

La librería consulta durante el día:

- una venta por número;
- las últimas diez ventas de un cliente;
- las últimas veinte ventas pagadas de una tienda;
- documentos cuya descripción contiene la palabra «reclamación».

Un diseño inicial razonable podría estudiar:

1. `{numero:1}` con unicidad si el número no se puede repetir;
2. `{clienteId:1,fecha:-1}`;
3. una combinación de `tienda`, `estado` y `fecha` según selectividad y ordenación;
4. un índice `text` sobre `descripcion`.

Qué pasaría si la aplicación cambia

Se añade ahora una pantalla que filtra únicamente por `fecha`, sin cliente, tienda ni estado. Ninguno de los índices compuestos anteriores garantiza resolver esa nueva pregunta óptimamente: su primer campo puede impedir un acceso selectivo directo por fecha.

Por tanto, el diseño no es una lista permanente. Debe revisarse al cambiar las consultas. Esa es la razón de mantener un inventario de operaciones reales y no solo un catálogo de campos.

Ejercicio oral

Elige una de las cuatro preguntas y explica por qué tu índice propuesto tiene sentido; identifica otra pregunta para la que probablemente no sería igual de útil. Haber entendido eso significa que ya estás razonando como diseñador y no como alguien que copia comandos.

Glosario mínimo

Base de datos: agrupación lógica de colecciones. **Colección:** conjunto de documentos. **Documento:** registro estructurado, con campos y valores. **Índice:** estructura auxiliar mantenida para determinadas operaciones. **Clave:** valor o conjunto de valores indexados. **Plan:** estrategia elegida para resolver una consulta. **COLLSCAN:** recorrido de colección. **IXSCAN:** recorrido de entradas de índice. **FETCH:** lectura de documentos. **Selectividad:** proporción de documentos que cumplen el filtro. **Cardinalidad:** cantidad de valores distintos. **TTL:** política de caducidad de documentos mediante índice de fechas.

Fuentes técnicas para profundizar

- Documentación de índices de MongoDB 8.0
- Resultados de explain en MongoDB 8.0
- Autenticación y seguridad

El siguiente documento es **Prácticas de índices**. Allí prepararás el entorno desde una terminal limpia, observarás cada resultado y aprenderás a diagnosticar los fallos más frecuentes.